> You are the Lumen Gen AI coach, and you have great teaching skills & deep knowledge of Lumen Gen AI platform, its capabilities, and its particular domain. Teach me this end to end with proper explanation, examples, illustration, etc. Make it super comprehensive. Below are the topics that you provided under this domain:
>
> 8 Agentic RAG & Multi-Agent Systems
> 8.1 Agentic RAG Concepts

Let's master Agentic RAG and Multi-Agent Systems. This is the domain that defines senior Gen AI engineer work — the shift from pipelines to autonomous reasoning systems.

The Fundamental Shift: Pipeline → Agent

In all the domains we've covered so far, RAG is a pipeline: a deterministic sequence of steps. Query arrives → retrieve → augment → generate → return. The flow is fixed. The system doesn't decide anything.

Agentic RAG introduces a loop: the LLM becomes a reasoning engine that decides what actions to take, observes results, and decides what to do next. The critical difference: the system has goals, plans, uses tools, evaluates its own progress, and adapt its strategy mid-flight.

Think of it this way: a pipeline RAG is a vending machine: you put in a query, out comes an answer. An agentic RAG is a research assistant: you give it a goal, it figures out what to search for, read the results, decide whether they have enough information, search again if needed, then synthesize a comprehensive answer.

The enabling mechanism is **tool use**: the LLM generates structured outputs that call external functions (retrieve, calculate, web search, code interpreter), receives results, and continues reasoning.

1. Agentic RAG Concepts — The Act Foundation

The canonical pattern for agents is **ReAct** (Reasoning + Acting). The agent alternates between two phases:

- **Thought:** the LLM reasons about what it knows and what it needs
- **Action:** the LLM calls a tool (retrieve, web search, calculator, etc.)
- **Observation:** the tool result is appended to context
- **Repeat** until the agent decides it has enough to answer

This is fundamentally different from a pipeline. The agent decides at each step whether to retrieve more, what to retrieve, and when to stop. Retrieval is conditional, iterative, and goal-directed.

Live ReAct trace — "What's the gross level dental rate for Lumen 6300 plans?"
Thought 1
I need to compare dental coverage 6300 monthly premium across plans. Let me start by finding dental plans in the knowledge base.
Action 1
search_knowledge_base(query="dental 6300", top_k=5)
Obs 1
Plan A: Full dental (Coverage: 75%), Plan B: Basic dental (Coverage: 50%), Plan C: dental premium add-on (Coverage: 80%)
Thought 2
Plan A has best coverage. Now I need premiums to find which plans are under 6300.
Action 2
search_knowledge_base(query="dental 6300 premium", top_k=5)
Obs 2
Plan A: $5,800/yr, Plan B: $3,600/yr, Plan C: $4,800/yr
Thought 3
Plan A is over 6300. Plan B is under 6300 but has limited dental. Plan C is under 6300 and has 80% coverage. Recommend Plan C as the best value option.
Answer: Plan C ($4,800/yr) is the only plan under 6300 with dental coverage. Plan B is cheaper ($3,600/yr) but has only 50% coverage.
The key insight from this trace: the agent made two targeted retrieval calls instead of one broad one. The second retrieval was informed by the first — the agent knew it needed premium data because it saw that dental coverage was insufficient. A pipeline would have to retrieve everything in one shot and likely miss one of the two required pieces.
Tool calling for retrieval is how modern LLMs implement this. Instead of special ReAct tokens, the LLM generates structured JSON that calls a registered tool:
system:
tools = [
{
"name": "retrieve_knowledge_base",
"description": "Search internal insurance documents for relevant information",
"parameters": {
"type": "object",
"properties": {
"query": {"type": "string"},
"filter_carrier": {"type": "string", "optional": true}
}
}
},
{
"name": "search_web",
"description": "Search the web for current information",
"parameters": {"type": "object", "properties": {"query": {"type": "string"}}}
}
]
The LLM decides which tool to call, generates the arguments, receives the result, and continues. This is the foundation of every agentic RAG system.

2. LangGraph RAG — State Machines as Agent Architecture

LangGraph is the production framework for building agentic RAG. It models the agent as a state machine: nodes are functions that read and write shared state, edges are transitions. The critical innovation over simple chains: **conditional edges** let the graph decide at runtime which node to visit next based on the current state.

Let me now show you the actual LangGraph code that implements this architecture, with all the key concepts.

State definition — the typed dict that flows through all nodes:

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.sqlite import SqliteSaver
import operator

class AgenticRagState(TypedDict):
    query: str
    rewritten_query: str
    classification: Literal["simple", "complex"]
    retrieved_chunks: Annotated[List[str], operator.add]
    generation: str
    grade: Literal["relevant", "irrelevant"]
    # Accumulate tool calls across iterations
    tool_calls: Annotated[List[dict], operator.add]
    messages: Annotated[List[BaseMessage], operator.add]
```

Conditional edge — the routing function:

```
def route_after_grading(state: AgenticRagState) -> str:
    # This function returns a node NAME, not data
    # The graph uses this to decide which node to visit
    if state.get("grade") == "relevant":
        return "generate"
    if state.get("rewritten_count", 0) >= 2:
        return "fallback_answer"  # Give up after 2 rewrites
    return "rewrite_query"  # Try rewriting
MAX_REWRITES = 2

def route_after_classification(state):
    return state["classification"]

rag_graph.add_conditional_edges(
    "classify_query",
    route_after_classification,
    {"simple": "simple_retrieve", "complex": "decompose_and_retrieve"}
)
rag_graph.add_conditional_edges(
    "grade_relevance",
    route_after_grading,
    {"generate": "generate", "rewrite_query": "rewrite_query", "fallback_answer": "fallback_answer"}
)
# All three non-conditional edges
rag_graph.add_edge("rewrite_query", "simple_retrieve")
rag_graph.add_edge("simple_retrieve", "grade_relevance")
rag_graph.add_edge("decompose_and_retrieve", "grade_relevance")
rag_graph.add_edge("generate", "human_review_checkpoint")
rag_graph.add_edge("fallback_answer", "human_review_checkpoint")
rag_graph.add_edge("human_review_checkpoint", END)
```

Full run parallel retrieval — Send API:

```
def fan_out_to_parallel_retrieve(state):
    # Send parallel retrieval for each knowledge source
    return [
        Send("retrieve_internal_kb", {"query": state["query"]}),
        Send("retrieve_web", {"query": state["query"]}),
        Send("retrieve_tool_db", {"query": state["query"]}),
    ]
# All three run concurrently, results merged into operator.add
```

Human-in-the-loop checkpoint + persistence:

```
from langgraph.checkpoint.sqlite import SqliteSaver

checkpointer = SqliteSaver(sqlite_conn_string=DB_URL)

# Interrupt before generation so human can review retrieved chunks
compiled = rag_graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["human_review_checkpoint"]
)

# Resume after human approval
config = {"configurable": {"thread_id": "session_123"}}
compiled.invoke(None, config=config)  # Continues from checkpoint
```

Four LangGraph concepts you must explain fluently in interviews:

State is the shared dict that every node reads from and writes to. The `Annotated[List, operator.add]` pattern is crucial for parallel nodes — it tells LangGraph to append (not overwrite) when multiple nodes write to the same field simultaneously.

Conditional edges are functions that return a node name. They're the decision points. The routing function receives the current state and decides where to go next. This is where all the "intelligence" of adaptive retrieval lives.

Checkpointers persist the entire graph state to a database (PostgreSQL, SQLite, Redis). This enables resumable execution — long-running agent tasks can be paused, the process can crash, and execution resumes from the last checkpoint. Critical for production.

Human-in-the-loop interrupts execution at a specified node, persists the state, and waits for human input before continuing. The human can inspect intermediate results, modify the state, and approve continuation. Use for high-stakes workflows (financial decisions, medical recommendations, legal documents).

3. Multi-Agent RAG — When One Agent Isn't Enough

A single agent is limited by its context window and specialization. Multi-agent systems route different parts of a complex task to specialized agents that each operate within their domain, then aggregate results.

The supervisor pattern is the most common multi-agent topology. The supervisor receives the full task, decomposes it into subtasks, assigns each to a specialist agent, and synthesizes the results. In LangGraph, each worker agent is a compiled subgraph — the supervisor calls them using `Command` returns.

Domain-scoped retrieval is the killer use case for multi-agent RAG. An insurance query that touches Plan X regulatory compliance and competitive positioning versus Industry Y needs three different retrieval strategies: vector search for conceptual matching, exact match for plan IDs, hybrid search for keyword-sensitive analysis. Rather than trying to handle all three with one agent, you route to specialists.

Cross-agent memory (shared `Postgresstore`) checkpoints research agents write to and read from the same store. If the Plan Compare agent retrieves "Plan A has HSA" and the Regulatory agent runs on this when assessing compliance implications.

Result aggregation is the supervisor's job: it receives all specialist outputs, resolves conflicts, and produces a coherent response. Conflicting sources (one agent says "covered", another says "excluded") need explicit handling. Always include source attribution so the user can verify.

4. Tool Use Patterns — The Agent's Toolbelt

Agentic search tool:

```
The retrieve as a callable function. Always expose filter parameters (carrier, date range, document type) so the agent can narrow its search. The agent decides what to filter based on context.
# Tool definition
search = {
    "name": "search_knowledge_base",
    "description": "Search internal insurance documents. Use for questions about specific plans, coverage, or regulations.",
    "parameters": {
        "query": {"type": "string"},
        "filter_carrier": {"type": "string", "optional": true},
        "top_k": {"type": "integer", "default": 5}
    }
}
```

Code execution:

```
The agent writes Python code for calculations, data transformations, or analysis. Sandboxed execution prevents abuse, and timeouts prevent runaway processes. Essential for numeric reasoning that LLMs are poorly at mentally.
# Tool definition
execute = {
    "name": "execute_code",
    "description": "Execute Python code. Use for calculations, data transformations, or analysis. Output must be printed to stdout.",
    "parameters": {
        "code": {"type": "string"},
        "timeout_seconds": {"type": "integer", "default": 30}
    }
}
```

Web search tool:

```
Fallback when the KB doesn't contain relevant information. Or when the query explicitly asks about recent events. Tavily, Perplexity, or Google Search APIs work well.
# Tool definition
web_search = {
    "name": "web_search",
    "description": "Search the web for current information. Use when the knowledge base doesn't have relevant information or for recent events.",
    "parameters": {
        "query": {"type": "string"},
        "recency_days": {"type": "integer", "optional": true}
    }
}
```

Memory tools:

```
Explicit read/write tools for the agent's long-term memory. Enables the agent to remember user preferences, prior conclusions, and learned patterns across sessions. Backed by Redis or PostgreSQL.
# Tool definitions
remember = {"name": "remember", "description": "Store a fact for later retrieval.", ...}
recall = {"name": "recall", "description": "Retrieve previously stored facts.", ...}
```

Web crawl tool:

```
Fetch and extract text from a URL. Useful when search results point to specific pages worth reading in full. Strip HTML, handle errors gracefully.
# Tool definition
crawl = {
    "name": "crawl_webpage",
    "description": "Fetch and extract text from a URL. Use when you need to read a specific page in full.",
    "parameters": {
        "url": {"type": "string"},
        "max_chars": {"type": "integer", "default": 10000}
    }
}
```

API integration:

```
Structured access to internal systems. Rate limits and auth handled by the tool, not the agent. The agent just sees a clean interface.
# Tool definition
claims = {
    "name": "get_claim_history",
    "description": "Retrieve claim history for a policyholder.",
    "parameters": {
        "policyholder_id": {"type": "string"},
        "date_range": {"type": "object", "optional": true},
        "status": {"type": "array", "optional": true}
    }
}
```

Tool selection principles:

- Clear descriptions matter more than clever prompts. The LLM reads the description to decide when to call the tool.
- Filter parameters let the agent narrow scope. Expose them when the agent should be able to make scoping decisions.
- Error handling should return useful messages, not crash. The agent can retry with different parameters if it gets a helpful error.
- Rate limits and auth belong in the tool, not in the agent. Keep complexity out of the reasoning loop.

5. Memory in Agents — The Four Types You Must Know

This is one of the most interview-dense topics. Human memory has different stores — episodic, semantic, procedural, working. Agent memory architectures mirror this.

Episodic memory stores past interactions and their outcomes. When a similar task arises, retrieve relevant episodes to inform the current approach. Example: "last time I asked about Plan X dental, I found that Plan Y was better." Enables learning from experience without retraining.

Semantic memory stores facts, preferences, and learned patterns. "User prefers HMO plans", "Plan X requires referrals for specialists", "Claims over $5k need pre-auth." Retrieved when relevant to the current query. Enables personalization and accumulated knowledge.

Procedural memory stores how to do things: tool definitions, prompt templates, few-shot examples. Retrieved when similar task type arises. Enables consistent behavior across similar tasks.

Working memory is the current context window: conversation history, retrieved chunks, tool results. Managed by truncation, summarization, or sliding window. Limited by model context length.

Memory-augmented agents combine all four. The agent retrieves relevant memories at the start of a task, accumulates new memories during the interaction, and consolidates important patterns at the end. Example: "User asks about dental → retrieve past dental queries → notice they always compare 6300 plans → store preference: user cares about 6300 dental → next time, proactively include 6300 comparison."

Memory tools let the agent control its own memory. Instead of automatic storage, the agent decides what to remember and what to retrieve. Explicit `remember(fact)` and `recall(query)` tools give the agent control over its cognitive architecture.

Memory consolidation prevents unbounded growth. Periodically summarize episodic memories into semantic facts, decay scores for rarely accessed memories, and forget what's no longer relevant. Production systems need this to avoid degrading retrieval quality over time.

6. Agent Orchestration Frameworks — The Competitive Landscape

LangGraph — production-grade stateful agents. Graph-based state machine. Maximum control over agent behavior. Human-in-the-loop and persistence built-in. Best for complex branching logic, long-running sessions, and production reliability. Steeper learning curve than alternatives.

CrewAI — role-based multi-agent collaboration. Define agents by role (researcher, analyst, writer) and assign tasks. Sequential or parallel execution. Simple abstraction for multi-agent workflows. Less control than LangGraph.

Claude Agent SDK — Anthropic's agent framework. Tight integration with Claude models. Tool use, handoffs, guardrails. Newer ecosystem, fewer examples.

Custom — roll your own. Full control over state, routing, and execution. Maximum flexibility. Expensive to maintain. Only justified when frameworks don't fit your use case.

Decision framework:

- Prototyping → CrewAI (fastest to working demo)
- Production complex workflows → LangGraph (most control, most battle-tested)
- Claude-native → Claude Agent SDK (if committed to Anthropic models)
- Maximum control → Custom (when frameworks add more friction than value)

Common mistake: Spending weeks fighting framework abstractions when a simple custom loop would work in days.

Handoff patterns in LangGraph: the `Command(goto="...", update={...})` return value is the primary handoff mechanism. An agent returns `Command(goto="research_agent", update={"findings": results})` to hand off to another agent with extracted state. The `Send(node, payload)` function passes execution to human review. The `END` constant terminates the graph.

Interview Question Bank — Domain 8

Basics

- **What is the ReAct pattern and how does it differ from a RAG pipeline?**

ReAct (Reasoning + Acting) interleaves thoughts (reasoning steps) with tool calls (actions) and observations (results). The model decides what to do next based on what it learned from previous steps. Pipeline RAG runs a fixed sequence: embed → retrieve → augment → generate. It doesn't decide whether to retrieve, what to retrieve, or when to stop. Agentic RAG uses ReAct: the LLM reasons about the query, calls a retrieval tool, observes the results, reasons about whether it has enough information, and repeats if needed. The critical difference is conditional, iterative, goal-driven retrieval vs. fixed single-shot retrieval.

- **What is a LangGraph StateGraph and what is the Agentic State?**

A LangGraph StateGraph is a directed graph where nodes are Python functions (processing steps) and edges are transitions between them. Conditional edges are functions that return a node name, implementing decision logic. The Agentic State is a typed dictionary that flows through all nodes. Fields can be overwritten (default) or accumulated (using `operator.add` for lists). Nodes read state, perform some operation, and return a dictionary of state updates. The `Annotated[List, operator.add]` pattern is fundamental for parallel execution — when multiple nodes write to the same field simultaneously, LangGraph knows to append rather than overwrite. The checkpointer persists the entire state to a database, enabling resumable execution after crashes or human review.

Mid-level

- **Explain the four memory types in agents and how you'd implement them in production.**

Short-term (in-context): the conversation history in the current session, stored in memory and passed to the messages list. Compressed with ConversationSummaryBufferMemory when ~20 turns to avoid context overflow. Episodic: past interactions and their outcomes, stored in a vector store. Retrieve relevant episodes when similar tasks arise. Enables learning from experience without retraining. Semantic: facts, preferences, and learned patterns. "User prefers HMO plans", "Plan X requires referrals for specialists". Retrieved by relevance to the current query. Procedural: how to do things. Tool definitions, prompt templates, few-shot examples. Retrieved when similar task type arises. Implementation: Redis or PostgreSQL for fast access, vector store for semantic retrieval, checkpointer for persistence across sessions. Retrieval strategy: recency (recent memories weighted higher), relevance (semantic similarity), importance (high-impact memories persist longer). Consolidation: periodically summarize episodic into semantic facts, decay scores for rarely accessed memories, forget what's no longer relevant.

- **How does LangGraph's human-in-the-loop checkpoint work?**

Human-in-the-loop pauses execution at a specified node, persists the entire state to a database, and waits for human input before continuing. When the graph reaches the node, it saves all state to the checkpointer (SqliteSaver, PostgresSaver, RedisSaver) and returns control to the caller. The caller displays the intermediate state to a human (e.g., The retrieved chunks, the proposed answer). The human can approve (call `.update(None, config=thread_config)` to resume), edit (modify state before resuming), or reject (abort the workflow). The checkpointer enables this because it decouples execution from persistence. Production considerations: Redis for low-latency checkpoints, PostgreSQL for audit trails, S3 for large state objects. Timeout handling: if human doesn't respond within N minutes, auto-escalate or fail gracefully.

Advanced

- **Design a multi-agent RAG system for an insurance broker platform where users can ask about plans, file claims, check regulatory compliance, and perform competitive market analysis.**

Architecture: Langgraph multi-agent system with a supervisor and four specialist agents. Supervisor receives query, classifies the intent, routes to specialist. Plan Compare specialist: retrieves plan documents, performs side-by-side comparisons, identifies coverage differences. Claims specialist: retrieves claim history, checks eligibility, calculates expected reimbursement. Regulatory specialist: retrieves compliance documents, checks state-specific requirements, flags potential issues. Web research specialist: searches for competitive intelligence, market data, recent news. Shared memory: Postgresstore namespaces for cross-agent context. Supervisor synthesizes outputs, resolves conflicts, produces coherent response. Human-in-the-loop: checkpoint before generation for review of retrieved chunks. Cross-session continuity: thread_id persists conversation history, user_id retrieves preferences and past interactions. Tools: hybrid search (vector + keyword), structured lookup (plan IDs, claim numbers), calculator, web search. Evaluation: RAGAS for retrieval quality, LLM-as-judge for response quality, human review for high-stakes decisions.

- **Your agent is stuck in a loop, calling the same tool with similar arguments repeatedly. How do you detect and prevent this?**

Detection: track tool calls in state, compare new calls to previous calls (exact match or semantic similarity). Prevention options: (1) Max iterations: hard limit on loop count, force exit after N iterations. (2) Diminishing returns: if last 3 retrievals returned similar chunks, the agent is probably stuck. (3) Progress tracking: is the agent getting closer to its goal? (4) Cost budget: stop when token spend exceeds threshold. Recovery: when loop detected, inject a system message forcing the agent to reconsider its approach, or route to a fallback handler, or escalate to human. Logging: log every tool call with arguments for post-hoc analysis. Production: alert on agents that hit iteration limits frequently — indicates systematic issue.

- **Compare LangGraph, CrewAI, and building custom agents. When would you choose each?**

LangGraph: production-grade stateful agents. Graph-based state machine. Maximum control over agent behavior. Human-in-the-loop and persistence built-in. Steeper learning curve. CrewAI: role-based multi-agent collaboration. Define agents by role (researcher, analyst, writer) and assign tasks. Sequential or parallel execution. Simple abstraction for multi-agent workflows. Less control than LangGraph. Custom: full control over state, routing, and execution. Maximum flexibility. Expensive to maintain. Only justified when frameworks don't fit your use case. Decision framework: Prototyping → CrewAI (fastest to working demo). Production complex workflows → LangGraph (most control, most battle-tested). Claude-native → Claude Agent SDK (if committed to Anthropic models). Maximum control → Custom (when frameworks add more friction than value). Common mistake: Spending weeks fighting framework abstractions when a simple custom loop would work in days.

- **Design a multi-agent RAG system for an insurance broker platform where users can ask about plans, file claims, check regulatory compliance, and perform competitive market analysis.**

Architecture: LangGraph multi-agent system with a supervisor and four specialist agents. Supervisor receives query, classifies the intent, routes to specialist. Plan Compare specialist: retrieves plan documents, performs side-by-side comparisons, identifies coverage differences. Claims specialist: retrieves claim history, checks eligibility, calculates expected reimbursement. Regulatory specialist: retrieves compliance documents, checks state-specific requirements, flags potential issues. Web research specialist: searches for competitive intelligence, market data, recent news. Shared memory: Postgresstore namespaces for cross-agent context. Supervisor synthesizes outputs, resolves conflicts, produces coherent response. Human-in-the-loop: checkpoint before generation for review of retrieved chunks. Cross-session continuity: thread_id persists conversation history, user_id retrieves preferences and past interactions. Tools: hybrid search (vector + keyword), structured lookup (plan IDs, claim numbers), calculator, web search. Evaluation: RAGAS for retrieval quality, LLM-as-judge for response quality, human review for high-stakes decisions.

- **Your agent is stuck in a loop, calling the same tool with similar arguments repeatedly. How do you detect and prevent this?**

Detection: track tool calls in state, compare new calls to previous calls (exact match or semantic similarity). Prevention options: (1) Max iterations: hard limit on loop count, force exit after N iterations. (2) Diminishing returns: if last 3 retrievals returned similar chunks, the agent is probably stuck. (3) Progress tracking: is the agent getting closer to its goal? (4) Cost budget: stop when token spend exceeds threshold. Recovery: when loop detected, inject a system message forcing the agent to reconsider its approach, or route to a fallback handler, or escalate to human. Logging: log every tool call with arguments for post-hoc analysis. Production: alert on agents that hit iteration limits frequently — indicates systematic issue.

- **Compare LangGraph, CrewAI, and building custom agents. When would you choose each?**

LangGraph: production-grade stateful agents. Graph-based state machine. Maximum control over agent behavior. Human-in-the-loop and persistence built-in. Steeper learning curve. CrewAI: role-based multi-agent collaboration. Define agents by role (researcher, analyst, writer) and assign tasks. Sequential or parallel execution. Simple abstraction for multi-agent workflows. Less control than LangGraph. Custom: full control over state, routing, and execution. Maximum flexibility. Expensive to maintain. Only justified when frameworks don't fit your use case. Decision framework: Prototyping → CrewAI (fastest to working demo). Production complex workflows → LangGraph (most control, most battle-tested). Claude-native → Claude Agent SDK (if committed to Anthropic models). Maximum control → Custom (when frameworks add more friction than value). Common mistake: Spending weeks fighting framework abstractions when a simple custom loop would work in days.

The Master Mental Model for Agentic RAG

A pipeline is a road. An agent is a driver.

The road has a fixed route: A → B → C → D. Traffic, weather, construction — the road doesn't care. The driver can take detours, stop for information, backtrack if the road is blocked, and take different routes based on current conditions.

LangGraph is the road network designer: you define which roads exist and which junctions allow turns. The LLM is the driver: it decides which turns to take at each junction.

The four production concerns that define senior from junior:

State design — what goes in. Agentic state has lists that accumulate (`operator.add`) vs overwrite. A poorly designed state causes race conditions in parallel nodes.

Conditional edge design — the routing functions are the intelligence of the system. Poorly designed routing creates infinite loops or dead ends. Always have escape conditions (max iterations, diminishing returns, confidence thresholds).

Checkpointer choice — SqliteSaver for development, PostgresSaver for production, RedisSaver for low-latency. Wrong choice means you can't resume after crashes or you can't scale.

Tool design — clear descriptions, filter parameters, error handling that returns useful messages. The LLM reads tool descriptions to decide when to call them.

You are out of free messages until 10:00 PM
Upgrade
Reply...
Sonnet 4.6
Claude is AI and can make mistakes. Please double-check responses.